

2026 암호분석경진대회

8번 문제

복원한 마스터키는 다음과 같다.

0x29 23 be 84 e1 6c d6 ae 52 90 49 f1 f1 bb e9 eb

1. notation

SB	Subbytes 연산, 역연산의 경우 SB^{-1} 로 표기
SR	ShiftRows 연산, 역연산의 경우 SR^{-1} 로 표기
MC	MixColumns 연산, 역연산의 경우 MC^{-1} 로 표기
ARK	AddRoundKey 연산
P	16 byte 평문
C	16 byte 암호문
K	16 byte 마스터키
k_i	i 번째 16-byte round key
F	$MC \circ SR \circ SB$, $F(X) = MC(SR(SB(X)))$
$X[i]$	16 byte X 값의 i 번째 byte 값
$C(X, i)$	16 byte X 을 4 x 4 state로 표현하였을 때, i 번째 열(워드)

2. leaked.txt 분석

leaked.txt의 “-”는 알 수 없는 비트이고, 그 외에는 노출된 비트이다.

상태	노출된 위치
X^0	16 바이트 전체
X^1	index 0 전체
X^2	index 0,1,2,3 : 각 1비트 마스킹
X^3	16바이트 전체 : 각 1비트 마스킹
X^4	index 0,5,10,15 : 각 1비트 마스킹
X^6	index 0, 2 전체
X^7	16 byte 전체

3. 마스터키 복구

1) $K[0]$ 복구

X^0 값을 모두 알고 있으므로, $K[0] = F(X^0) \oplus X^1[0]$ 을 통해 구할 수 있다. $K[0] = 0x09 \oplus 0x20 = 0x29$

2) $K[6], K[7], K[8], K[9]$ 에 대한 관계식 추출

X^3 후보를 통해 round 2 값을 역추적하기 위해 $A_i = MC^{-1}(C(X^3, i))$ 을 계산한다. X^3 을 계산하기 위해 사용된 2라운드 키는 다음과 같이 도출되었으므로,

$$k_2 = MC(K[6] \parallel K[10] \parallel K[5] \parallel K[12] \parallel K[3] \parallel K[2] \parallel K[15] \parallel K[9] \parallel K[1] \parallel K[4] \parallel K[8] \parallel K[11] \parallel K[13] \parallel K[7] \parallel K[0] \parallel K[14])$$

A_i 값은 다음과 같이 표현될 수 있다.

$$A_0 = \begin{pmatrix} SB(X^2[0]) \oplus K[6] \\ SB(X^2[5]) \oplus K[10] \\ SB(X^2[10]) \oplus K[5] \\ SB(X^2[15]) \oplus K[12] \end{pmatrix}, A_1 = \begin{pmatrix} SB(X^2[4]) \oplus K[3] \\ SB(X^2[9]) \oplus K[2] \\ SB(X^2[14]) \oplus K[15] \\ SB(X^2[3]) \oplus K[9] \end{pmatrix}, A_2 = \begin{pmatrix} SB(X^2[8]) \oplus K[1] \\ SB(X^2[13]) \oplus K[4] \\ SB(X^2[2]) \oplus K[8] \\ SB(X^2[7]) \oplus K[11] \end{pmatrix}, A_3 = \begin{pmatrix} SB(X^2[12]) \oplus K[13] \\ SB(X^2[1]) \oplus K[7] \\ SB(X^2[6]) \oplus K[0] \\ SB(X^2[11]) \oplus K[14] \end{pmatrix}$$

$X^2[0], X^2[1], X^2[2], X^2[3]$ 값은 leaked.txt를 통해 노출된 값으로 각각 2개의 후보뿐이므로, 이를 고정하면 $K[6], K[7], K[8], K[9]$ 을 복원할 수 있다.

3) $K[5], K[8], K[10], K[15]$ 에 대한 관계식 추출

X^2 의 첫 column 후보를 이용하여 round 1을 역추적한다. 다음과 같은 관계식 B_0 을 얻을 수 있다.

$$B_0 = MC^{-1} \begin{pmatrix} X^2[0] \\ X^2[1] \\ X^2[2] \\ X^2[3] \end{pmatrix} = \begin{pmatrix} SB(F(X^0)[0] \oplus K[0]) \oplus K[5] \\ SB(F(X^0)[5] \oplus K[5]) \oplus K[8] \\ SB(F(X^0)[10] \oplus K[10]) \oplus K[6] \\ SB(F(X^0)[15] \oplus K[15]) \oplus K[3] \end{pmatrix}$$

첫 번째 식에서 $K[5]$ 가 결정된다. 두 번째 식은 2)에서 계산한 $K[8]$ 과 일치해야 하므로 강한 필터가 된다. 세 번째 식은 $K[10]$ 의 후보를 제한하고, 네 번째 식은 추후 $K[3]$ 을 고정한 뒤 $K[15]$ 을 계산하는데 사용한다.

4) $K[10], K[11], K[12], K[13]$ 에 대한 관계식 추출

k_3 는 마스터키 순열이 그대로 들어가는 라운드키다. k_3 의 index 0, 5, 10, 15 위치에는 각각 $K[10], K[11], K[12], K[13]$ 이 들어간다. 따라서 $C = F(X^3)$ 을 계산하면 leaked.txt의 X^4 1비트 마스킹 대각선 누출로 $K[10], K[11], K[12], K[13]$ 에 대한 관계식을 얻을 수 있다. 누출된 X^4 의 대각선 값들은 각각 1bit 마스킹으로 인한 2개의 후보를 가지므로 해당 마스터키들 역시 2개의 후보를 가진다. $K[10]$ 의 경우, 3)의 B_0 식으로 다시 필터링한다.

$$\begin{aligned} K[10] &= X^4[0] \oplus C[0] \\ K[11] &= X^4[5] \oplus C[5] \\ K[12] &= X^4[10] \oplus C[10] \\ K[13] &= X^4[15] \oplus C[15] \end{aligned}$$

5) 남은 마스터키 $K[1], K[2], K[3], K[4], K[14], K[15]$ 관계식 추출

현재까지 $K[0], K[5], K[6], K[7], K[8], K[9], K[10], K[11], K[12], K[13]$ 이 정해진다. 2)의 A_i 식을 사용하면 X^2 의 여러 unknown byte들도 역 Subbytes 연산을 통해 구할 수 있다.

$$\begin{aligned} X^2[5] &= SB^{-1}(A_0[1] \oplus K[10]) \\ X^2[10] &= SB^{-1}(A_0[2] \oplus K[5]) \\ X^2[15] &= SB^{-1}(A_0[3] \oplus K[12]) \\ X^2[7] &= SB^{-1}(A_2[3] \oplus K[11]) \\ X^2[12] &= SB^{-1}(A_3[0] \oplus K[13]) \\ X^2[6] &= SB^{-1}(A_3[2] \oplus K[0]) \end{aligned}$$

이제 $K[3]$ 을 1-byte 전수 조사를 통해 0부터 255까지 탐색한다. $K[3]$ 가 정해지면 $X^2[4]$ 가 정해지고, $C(X^2, 1)$ 에 MC^{-1} 을 적용하여 B_1 을 만든다.

$$B_1 = MC^{-1}(C(X^2, 1)) = MC^{-1} \begin{pmatrix} X^2[4] \\ X^2[5] \\ X^2[6] \\ X^2[7] \end{pmatrix}$$

$B_1[3]$ 로 $K[3]$ 을 먼저 검증하고, 나머지 식에서 $K[14], K[15], K[4]$ 을 계산한다.

$$\begin{aligned} CHECK: B_1[3] &= SB(F(X^0)[3] \oplus K[3]) \oplus K[12] \\ K[14] &= F(X^0)[14] \oplus SB^{-1}(B_1[2] \oplus K[13]) \\ CHECK: B_1[1] &= SB(F(X^0)[9] \oplus K[9]) \oplus K[14] \\ K[15] &= F(X^0)[15] \oplus SB^{-1}(B_0[3] \oplus K[3]) \\ K[4] &= F(X^0)[4] \oplus SB^{-1}(B_1[0] \oplus K[15]) \end{aligned}$$

이후 $C(X^2, 3)$ 을 통해 $K[1]$ 을 구한 뒤 이전 관계식들을 만족하는지 확인한다.

$$\begin{aligned}
X^2[11] &= SB^{-1}(A_3[3] \oplus K[14]) \\
X^2[13] &= SB^{-1}(A_2[1] \oplus K[4]) \\
X^2[14] &= SB^{-1}(A_1[2] \oplus K[15]) \\
B_3 &= MC^{-1} \begin{pmatrix} X^2[12] \\ X^2[13] \\ X^2[14] \\ X^2[15] \end{pmatrix}
\end{aligned}$$

$$\begin{aligned}
CHECK : B_3[0] &== SB(F(X^0)[12] \oplus K[12]) \oplus K[0] \\
K[1] &= F(X^0)[1] \oplus SB^{-1}(B_3[1] \oplus K[4]) \\
CHECK : B_3[2] &== SB(F(X^0)[6] \oplus K[6]) \oplus K[7] \\
CHECK : B_3[3] &== SB(F(X^0)[11] \oplus K[11]) \oplus K[9]
\end{aligned}$$

마지막으로 $K[2]$ 에 대한 1-byte 전수 조사를 통해 $C(X^2, 2)$ 의 네 식을 모두 만족하는 값만 남긴다. 특히 $B_2[2]$ 에 서는 $K[2]$ 가 S-box 입력과 key xor 양쪽에 동시에 들어가므로, 단순 선형식이 아닌 256개의 검색으로 처리하였 다.

1. $X^2[8] = SB^{-1}(A_2[0] \oplus K[1])$
2. For $K[2]$ in 0 ... 255 Do
3. $X^2[9] = SB^{-1}(A_1[1] \oplus K[2])$
4. $B_2 = MC^{-1} \begin{pmatrix} X^2[8] \\ X^2[9] \\ X^2[10] \\ X^2[11] \end{pmatrix}$
5. CHECK : $B_2[0] == SB(F(X^0)[8] \oplus K[8]) \oplus K[11]$
6. CHECK : $B_2[1] == SB(F(X^0)[13] \oplus K[13]) \oplus K[10]$
7. CHECK : $B_2[2] == SB(F(X^0)[2] \oplus K[2]) \oplus K[2]$
8. CHECK : $B_2[3] == SB(F(X^0)[7] \oplus K[7]) \oplus K[1]$

4. 후보 수 감소와 계산량

아래 표는 실제 후보 수 감소 과정이다. 누출된 unknown bit 수만 보았을 경우, 처음에는 $2^{16} \times 2^4$ 조합이지만, column 단위 MC^{-1} 관계식이 키 바이트의 후보 수를 빠르게 줄임을 확인하였다.

단계	남은/통과한 후보 수	설명
X^3 후보	2^{16}	16바이트 전체 : 각 1비트 마스킹
X^3 후보와 $X^2[0], \dots, X^2[1]$ 후보 조합	$2^{16} \times 2^4$	index 0,1,2,3 : 각 1비트 마스킹
$K[8]$ 관계식 통과	2^{12}	round 1과 round 2을 통해 동일한 $K[8]$ 을 얻는지 확인
$K[10]$ 관계식 통과	2^8	X^4 대각선 후보와 $B_0[2]$ 식을 동시에 만족
$K[11], K[12], K[13]$ 후보 확장	2^9	각각 최대 2개 후보
$K[3]$ 1차 조건 통과	494	$B_1[3]$ 식 만족
B_1 추가 조건 통과	3	$K[14], K[15], K[4]$ 결정 후 검증
B_3 조건 통과	1	$K[1]$ 결정
$K[2]$ 및 최종 leak/ciphertext 통과	1	유일한 마스터키 후보

결과적으로, 각 라운드 방정식이 서로 다른 경로로 같은 키 바이트를 2번 계산하도록 하여 우연히 일치할 확률을 2^8 만큼씩 줄여냄으로써, 공격에서 탐색하는 후보 수를 빠르게 줄여나갈 수 있었다. 최종적으로 얻어낸 마스터 키는 다음과 같으며, 해당 마스터키는 주어진 50000개의 평문-암호문 쌍을 동일하게 생성해냄을 확인하였다.

```
0x29 23 be 84 e1 6c d6 ae 52 90 49 f1 f1 bb e9 eb
```

5. 코드 구현

python으로 구현하였으며 풀이 코드는 외부 라이브러리 없이 Python 표준 라이브러리만 사용하였다. AES S-box와 inverse S-box를 테이블로 두고, $GF(2^8)$ 곱셈은 xtime 기반으로 구현하였다. MixColumns와 InvMixColumns 계수는 표준 AES와 동일하다.

함수	역할
mix_single_col / inv_mix_single_col	한 column에 표준 AES MixColumns / inverse MixColumns 적용
shift_rows	column-major state 기준 표준 AES ShiftRows
core	함수 F
round_keys	문제의 마스터키 순열과 MC 적용을 반영해 $k_0, \dots, k_6$ 생성
read_leak	0/1/- 패턴을 가능한 byte 후보 집합으로 변환
recover_key	본 문제 풀이의 후보 열거 및 관계식 검증 수행
encrypt_block	최종 후보 검증용 7라운드 암호화

함수 실행 결과는 다음과 같다.

```
$ python solve.py
master key = 2923be84e16cd6ae529049f1f1bbe9eb
verified 50000/50000 plaintext-ciphertext pairs
```